Lecture 2 Process Management

2022年9月14日 22:40

Process ≠ program

A program is just a piece of code.
- High-level language (C, C++, Java, …)
- Intermediate language (Java bytecode, LLVM IR, .NET CIL, …)
- Low-level language (assembly)
- Machine code

Life of a C program

Preprocessor (cpp)

The preprocessor expands **directives** such as *#include*, *#define*, *#ifdef*, … in the C source code.
gcc -E main.c shows the **expanded code**.

Compiler (gcc)

The compiler takes the expanded C code, checks the syntax, and generates assembly code (gcc) or LLVM IR (clang).
In the meantime, it also **optimizes** the code.
gcc -S stupid.c shows the assembly code without optimization.
gcc -S -O2 stupid.c shows the assembly code with optimization. 2 specifies the level of optimization. Optimization is by default turned off, because it makes debugging the source code difficult.

Assembler (as)

The assembler converts the generated assembly code to an **object file**. The object file contains machine code, but is not yet executable.
as hello.s -o hello.o shows the contents in the object file.

Linker (ld)

The linker combines object files and libraries to produce the **executable file**. A library is just a collection of functions and variables.
- Static linking: the final program embeds all library function used.
- Dynamic linking: the final program does not embed the library functions. The OS will load dynamic libraries when the program runs.

Processes

The **process** is the most central concept in an operating system.
- It is an abstraction of a running program.
- It attaches to all the memory that is allocated for the process.
- It associates with all the files opened by the process.
- It contains accounting information such as its owner, running time, memory usage…

Process identification

The OS gives each process a unique identification number, the **Process ID (PID)**.
- *getpid()* returns the PID of the calling process.
- *getppid()* returns the PID of the parent of the calling process.

Remark: if we run program in the shell multiple times, PID changes but PPID does not change because the parent is the shell.

Process creation

Before you call *fork()*, you have only one parent process. After calling *fork()*, the parent process forks into itself and a child process.

Example:

```
int main() {
  printf("Before fork, my PID is %d\n", getpid());
  fork();
  printf("After fork, my PID is %d\n", getpid());
}
```

fork.c

The second line will be printed twice, because the child process does not run from the beginning, and after forking both parent and child execute the same code after it.

The *fork()* function actually returns an integer value, 0 in child process and the pid of child in parent process (nonzero).

The child will inherit the program code (same code), the program counter (execute from same location), memory (variables and allocated memory), and opened files. However, the following are different: return value of *fork()*, pid, parent, running time (reset to 0), and file lock (do not inherit).

Example of special case:

```
int main() {
  printf("Hello ");
  fork();
  printf("CS202\n");
}
```

fork_buffer.c

printf() actually has a buffer in the file structure to reduce system calls, and this is inherited by the child. STDERR is unbuffered (*write()* immediately), but STDIN and STDOUT are line-buffered (*write()* at newline character). Hence, the output of this program will in fact be two lines of "Hello CS202". *setvbuf()* changes the buffering strategy, and *fflush()* can force a write. For example, adding *fflush(stdout)* before *fork()* will result in second line containing only "CS202".

Process execution

execve() and the *exec*()* family of functions: replaces the current process image with a new program. When called, the code is replaced and it will never return to the original code. Variables, allocated memory, and registers (such as program counter) are discarded, but pid, process relationship, and running time are preserved.

- Path name or file name
  - Default: path name
  - Functions with *p*: file name (*execlp()*, *execvp()*)
- Argument list or array
  - Functions with *l*: list (*execl()*, *execlp()*, *execle()*), in fact variadic function ending with NULL argument
  - Functions with *v*: array (*execv()*, *execvp()*, *execve()*)
- Environment variables
  - Default: inherit the current environment
  - Functions with *e*: specify a new environment array

Note: environment variables are a set of strings maintained by the shell, contained in *char **envp*. For Lab 2, we are only interested in the PATH environment variable.

Process creation and execution

system() is implemented using *fork()* and the *exec*()* family. We can fork a child process and let it run. However, we cannot control the OS's scheduling so that the child runs first. However, we can let the parent pause while child is running.

The *wait()* system call suspends the calling process until one of its children terminates. The *waitpid()* function waits for a particular child or a stopped/resumed child.

```
int my_system(const char *command) {
  if (fork() == 0) {
    execl("/bin/sh", "/bin/sh", "-c", command, NULL);
    exit(-1);
  }
  wait(NULL);
  return 0;
}

int main() {
  printf("Before system\n");
  my_system("ls");
  printf("After system\n");
}
```

Can we remove this exit()?

my_system.c

Note that we cannot remove the *exit()*, since *execl()* may fail, and we must imitate its behavior to exit the current code.

Processes in the kernel

Kernel-space memory

The memory is divided into user-space memory and kernel-space memory.
- User-space memory stores program code, process's memory, etc.
- Kernel-space memory stores kernel code, kernel data structures, loaded device drivers, etc.

In the kernel-space memory, the **process control block** contains everything related to a process.
- Doubly-linked list to store processes using *struct task_struct*
- Each process information contains pid, running time, file descriptors, etc.
- File descriptors is an array of opened files (0: STDIN, 1: STDOUT, 2: STDERR, 3+: yours)

Redirection: use *dup2()* function to duplicate the file descriptor, then use *close()* to close the unused file descriptor.

```
int main() {
  printf("Hello CS202\n");

  int fd = open("output.txt",
                O_CREAT|O_WRONLY|O_TRUNC,
                S_IRUSR|S_IWUSR);
  dup2(fd, 1);  // duplicate the file descriptor
  close(fd);    // close the unused file descriptor

  printf("Hello CS202 again\n");
}
```

redirect.c

Process execution

A process switches its execution from user mode to kernel mode by invoking system calls. When the system call finishes, the execution switches from kernel mode back to user mode.

Handling system calls

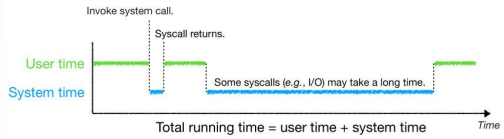
The CPU is running a process in user mode. At some time it wants to invoke the *getpid()* system call. Each system call has a unique **syscall number**, so the process puts the syscall number of *getpid()* in a specific CPU register (*e.g.*, *%rax*), and then it executes ***TRAP*** instrution to switch from user mode to kernel mode.

The kernel starts execution at the **syscall dispatcher.** It examines the syscall number, looks up the syscall table (mapping syscall number to syscall handler), and invokes the corresponding **syscall handler**.

The *sys_getpid()* handler reads the process ID of the calling process from **task_struct**, then it executes a ***RETURN-FROM-TRAP*** instruction to switch from kernel mode back to user mode.

Process time

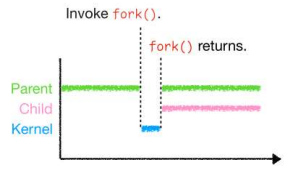
The execution of a process is divided into two parts: user time and system time.



The *time ./program* command can display the real, user, and sys time. Note that system calls can play a major role in performance. Some system calls can even stop your process until the data is available. Hence, it is important to reduce the number of system calls in a program for better performance.

Process-management syscalls in the kernel

fork()

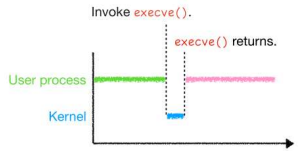


In kernel space memory:
- Create new process (new PID, running time reset to 0, clone file descriptors, etc.)
- Parent's list of children adds this new process, and this new process sets its parent.

In user space memory:
- Copy-on-write: if nothing writes to the new process, then user memory has a view of the new process but do not actually clone it. Clone only happens on writing.

If two processes sharing the same file write to that file together (e.g., maybe one has opened a file and forked before closing it), then the writing order is random. One needs to synchronize writing himself.

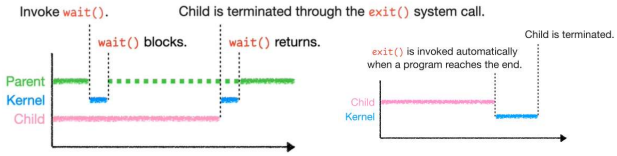
execve()



In kernel space memory:
- Everything of the process is preserved, and no processes are created.

In user space memory:
- The kernel loads the code of the new program and replaces the text.
- The kernel resets the memory (heap, stack, global variables, etc.)
- CPU registers (e.g. program counter) will be reset.

wait() and *waitpid()*



When calling *exit()*, the kernel frees all allocated memory and closes all opened files in kernel space memory. It also removes everything about the process from the user space memory.
The child's storage in the system is now minimal. However, it still owns the PID and *task_struct*, and the child now enters the **zombie state**.
Finally, the kernel sends the parent of the calling process a SIGCHLD signal.

Now, the parent is running concurrently with the child, and the SIGCHLD signal will be sent to parent. By default, a process **ignores the SIGCHLD signal** unless it has called *wait()* or *waitpid()*.

In detail, when *wait()* is called, the kernel sets a **signal handler** to the process, which will be executed when SIGCHLD arrives (note that this is a function pointer). Then, the kernel sets the process to the **blocked** (a.k.a. **sleeping**) state. Then, when SIGCHLD arrives, the parent's signal handler (not the original program code) is invoked. The signal handler lets the parent process **remove the signal** and **destroy the corresponding child process**. Finally, the signal handler is removed, so the process is once again ignoring SIGCHLD. It then returns to the previously-executing original program code **in the user space**. Therefore, it looks as if *wait()* is returned from its invocation.

Zombies

The zombie process lives up to the moment the parent process calls *wait()*. One should never leave any zombies in the system, and *wait()* and *waitpid()* are to reap zombie children. Zombies take up PIDs, so it is something regarding system resource management.

Signals

Signal is a form of **inter-process communication (IPC)**. A process can send signal to another process. When a signal arrives, the OS interrupts the process's normal control flow and executes the **signal handler**.

How are signals generated?

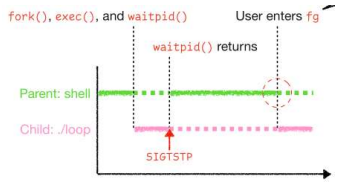
From the user space
- Keyboard: Ctrl-C sends SIGINT, Ctrl-Z sends SIGTSTP, Ctrl-\ sends SIGQUIT, etc.
- Commands: *kill*, *top*, etc.
- Using the *kill()* system call.

From the kernel or hardware
- SIGCHLD (child stopped or terminated) comes from the kernel.
- SIGFPE (floating point exception, e.g., division by zero) comes from the CPU.
- SIGSEGV (invalid memory reference, a.k.a. segmentation fault) comes from the CPU to the kernel and then to the process.

The *kill* command

The *kill* command sends SIGTERM to the target process by default. The default signal handler of SIGTERM is to terminate the process. For example, *kill 2250* terminates the process with PID 2250.

Actually, the *kill* command can send any signal. For example, *kill -TSTP 2250* suspends the process with PID 2250. The Linux shell will report something like *[1]+ Stopped ./program*. Also, *kill -CONT 2250* resumes this program.



Note that if simply a SIGCONT signal is sent to the child process instead of entering *fg* command, the child and the parent processes will in fact run concurrently, and the child process is running as a background job.

Just like the *kill* command, the *kill()* syscall **sends a signal** to a process in the format *int kill(pid_t pid, int sig)*. Also, the *raise()* library function sends a signal to yourself.

Signal	Description	Default handler
SIGINT	Interrupt from keyboard (Ctrl-C).	Terminate the process.
SIGTERM	Termination (the default signal sent by the kill command).	Terminate the process.
SIGKILL	Kill, no matter what.	Terminate the process (cannot be overridden).
SIGTSTP	Stop typed at terminal (Ctrl-Z).	Stop the process.
SIGCONT	Continue if stopped.	Continue the process.
SIGCHLD	Child stopped or terminated.	Ignore the signal.
SIGFPE	Floating point exception.	Terminate the process and dump core.

SIGTSTP	Stop typed at terminal (Ctrl-Z).	Stop the process.
SIGCONT	Continue if stopped.	Continue the process.
SIGCHLD	Child stopped or terminated.	Ignore the signal.
SIGFPE	Floating point exception.	Terminate the process and dump core.
SIGSEGV	Invalid memory reference (segmentation fault).	Terminate the process and dump core.

Changing signal handlers

*void (*signal(int sig, void (*handler)(int)))(int);*
Here, *type_a(*something)(type_b)* means a function pointer that takes arguments of *type_b* and returns value of *type_a*.
Hence, this can be simplified to *funcptr_signal(int sig, funcptr_handler)*, where both function pointers takes a single integer argument and returns void.
The returned function pointer is the old signal handler, and the one in the argument is the new signal handler.

Example: change the behavior of Ctrl-C

```
void handler(int sig) {
     // Do something  // The behavior of the new signal handler
}

int main() {
     signal(SIGINT, handler);  // register a new signal handler meeting Ctrl-C
     // Do something
}
```

The *pause()* system call puts the process to **block (sleep)** until the delivery of a signal handled by the process or a signal terminating the process. This is a special case for the *alarm()* system call, which sets up a **one-time asynchronous timer** for the process. A SIGALARM signal will be set to the process at timeout, default to terminate the process but can be overridden. On the other hand, the *setitimer()* system call sets up an **interval timer**. These functions can be used to implement *sleep()*.

Summary

Signal is a kind of **software interrupt**, which has many quirky behaviors. *kill()* is **not intended to kill anybody**, but to send signals. When in doubt, refer to the Linux manual pages.

Process organization

Booting the computer

BIOS locates the boot device. The boot code on the boot device is then executed, and the OS is started.

BIOS (Basic input/output system) is **firmware** (i.e., software providing low-level control for hardware). It is stored in a ROM chip or flash memory* on the motherboard.
*Flash memory: most of the time unchanged, only changed occasionally when for example firmware update.

The BIOS starts when the computer boots up. It checks all devices attached to the computer and locates the **boot device**. (Recent systems have replaced BIOS with UEFI, Unified Extensible Firmware Interface, which is a complete boot manager.)

Boot device

The boot code in the boot device's first sector is executed. The boot code reads the **partition table** and determine the **bootable partition**. The **boot loader** from the bootable partition is executed.

The boot loader locates and boots the **kernel image**. The kernel image is just a file (Linux: /boot/vmlinuz-*; macOS: /System/Library/Kernels/kernel; Windows: C:\windows\system32\ntoskrnl.exe). The kernel initializes the memory layout, device drivers, etc., and creates the **first** process.

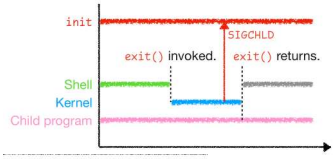
During booting, the kernel creates the first process (PID=1): /sbin/init.
- Recent Linux systems have replaced *init* with *systemd*.

init creates more processes using *fork()* and *execve()*.
- It reads configurations from /etc/rc.d about which programs to run.
- The entire **process tree** can be viewed using the *pstree* command.
- *init* continues running in the background **until the system is shut down**.

Orphans

A process becomes an **orphan** when its parent process terminates. The *init* process automatically adopts all orphaned processes (this is called **reparenting**).

In the kernel, the killed parent process (shell) becomes a zombie, and the child is reparented to *init* process.




```
void handler(int sig) {
  waitpid(…);
}

int main() {
  …
  signal(SIGCHLD, handler);
  …
  // doing other work
  …
}
```
init.c

Observations

Processes in Linux are organized as a **single tree**.
- Windows does maintain a forest-like process hierarchy, but we will focus on Unix/Linux.

Reparenting allows processes to run **without a parent shell**.
- Therefore, background jobs can survive even after the shell exits.

Process scheduling

Computers often do several things concurrently, even if it has only one CPU.

- It's called **multiprogramming**.

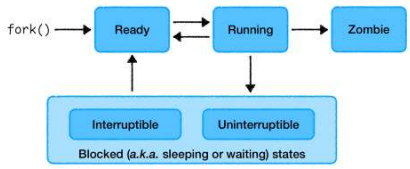
The CPU switches from process to process quickly, running each for a few milliseconds.

- It's called **multitasking**.

The OS needs to choose which process to run next.

- It's called **scheduling**.
- The part of the OS that makes the choice is called the **scheduler**.

Process states



Ready means **runnable but temporarily stopped** to let another process run.

- It is just created by *fork()*;
- It has run on the CPU for long enough, and the scheduler picks another process to run;
- It returns from a blocked state.

Running means the process is actually **using the CPU** at that instant.

- The scheduler picks this process to run.

Blocked means **unable to run** until some **external event** happens.

- It invokes the *pause()* system call;
- It reads from an I/O device;
- Etc.

Example: reading a file on the disk. The process has to wait for the response from the disk drive, so it is **blocked**. However, this blocked state is **interruptible**. For example, pressing Ctrl-C can get the process out of the blocked state.

Example: a blocked state may also be **uninterruptible**. For example, a process may need to wait for a resource, but it does not want to be disturbed while it is waiting. This scenario often happens within the kernel.

Zombie means the process **exits** and its **parent has not yet waited** for it. It terminates when the parent finally waits for it.

Context switching

The **scheduler** in the kernel will choose the next process to run. Before the scheduler can take up the CPU, it has to **back up register values**. The backup should be stored in the kernel space memory since the kernel space scheduler handles that. The register values are saved using *task_struct*. The **context** of a process consists of its user-space memory and its register values.

Then, the scheduler decides which process to run next. If the next process is different from the current one, the OS performs a **context switch**, including:

- Saving and restoring registers;
- Switching memory maps;
- Flushing and reloading the cache;
- Etc.

Context switching may be expensive (and the system is not doing any useful work).

Process scheduling

Most processes' execution **alternates between CPU execution and I/O wait**.

- **CPU-bound processes** spend most of their running time on the CPU. Examples: compiling a program, rendering a video, mining Bitcoin, scientific programming, etc. *time* command will show user time > sys time.
- **I/O-bound processes** spend most of their running time on I/O. Examples: /bin/ls (mainly disk I/O), downloading a file from the Internet, printing an image, etc. *time* command will show sys time > user time.

When to schedule

- A new process is created. It's up to the scheduler to decide whether to run the parent or the child.
- An existing process is terminated. The scheduler should choose another process to run. If no process is ready, and "idle" process is run.
- A process starts waiting for I/O (or something else). The process is **blocked** and thus the process should choose another process to run.
- A process finishes waiting for I/O. The process becomes **ready** to run again. It's up to the scheduler to decide whether to run it.
- Just periodic scheduling…

Nonpreemtive vs. preemptive scheduling

Nonpreemptive scheduling: when a process is scheduled, it keeps running until it starts **waiting** for the I/O (or something else), or it **voluntarily** relinquishes the CPU (*man 2 sched_yield*: *sched_yield()* causes the calling thread to relinquish the CPU. The thread is moved to the end of the queue for its static priority and a new thread gets to run.)

Preemptive scheduling:

- A process can run for a particular period of time.
- When the time is up or some particular events occurs (e.g., I/O completion), the process is **suspended** and another process may run.
- The **hardware clock interrupt** gives control of the CPU back to the scheduler.

General goals

Fairness: each process should have a fair share of the CPU.
Policy enforcement: the system's **policies** should be carried out. For example, the admin may state that certain processes have a higher priority.
Balance: all parts of the system should be kept busy all the time. For example, CPU-bound and I/O-bound processes should be mixed together.

Batch systems

A **batch system** collects a set of jobs and then process them **in batches**. Examples: bank systems, data analytics, etc. (or even your laundry basket, where you "submit" you dirty clothes at the end of each week).

Batch jobs can run without user interaction, so nonpreemptive scheduling or preemptive scheduling with large time slices are acceptable.

Scheduling goals for batch systems:

- High throughput: maximize the number of jobs per hour.
- Short turnaround time: minimize time between submission and completion.
- High CPU utilization: keep the CPU busy all the time.

Interactive systems

An **interactive system** may be a computer with an interactive user, or a server that serves multiple interactive users.

Preemption is essential to keep one process from denying service to others.

Scheduling goals for interactive systems:
- Short response time: minimize the time between request and response. Interactive jobs may take precedence over background jobs.
- Proportionality: meet users' expectations. You think it's OK to load a video game for several minutes, but not OK if it reacts a second after you press 'A'.

Real-time systems

Real-time systems must guarantee response within specified time constraints. Example: a robot welding cars moving down an assembly line.

Scheduling goals for real-time systems:
- Meeting deadlines: avoid losing data. If a device produces data at a regular rate, you must run the data-collection process on time.
- Predictability: avoid quality degradation (in multimedia systems). If the audio process runs too erratically (not regularly), the sound quality will deteriorate rapidly.

Scheduling algorithms *(... continued process scheduling)*

- First-come, first-served: it's nonpreemptive.
- Shortest job first: it can be nonpreemtive or preemptive.
- Round-robin: it's preemptive.
- Priority scheduling: it can be nonpreemptive or preemptive.

Metrics

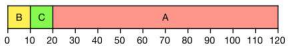
Wait time: the duration that the job is in the system but **not running**.
Turnaround time: the duration from when the job arrives in the system to the time it completes.

First-come, first-served (FCFS)

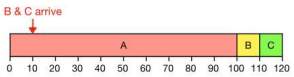
First-come, first-served (FCFS), a.k.a **first-in, first-out (FIFO)**, is a nonpreemptive scheduling algorithm in batch systems.

Nonpreemptive shortest job first (SJF)

Job	Arrival time	CPU requirement
A	0	100
B	0	10
C	0	10



Job	Arrival time	CPU requirement
A	0	100
B	10	10
C	10	10



The first case is good, but the second case is not good. Even though B and C arrived shortly after A, they have to wait until A completes. This is called the **convoy effect**, where a number of relatively short potential consumers of a resource get queued behind a heavyweight resource consumer.

Preemptive shortest job first (PSJF)

To address the previous concern, whenever a new job arrives, the **PSJF** scheduler determines which job has the **shortest remaining time**, and schedules that one.

Round-robin (RR)

PSJF looks great for batch systems. However, in interactive systems, users would demand a short response time, which is defined as the duration from when the job arrives in the system to **the first time it is scheduled**.

Round-robin (RR) is a preemptive scheduling algorithm in interactive systems. Each job is assigned a **time slice** (a.k.a **quantum**). The time slice is the amount of time the job is allowed to run. At the end of the time slice, the CPU is preempted and given to the next job. For simplicity, assume all jobs have the same time slice. Jobs are running one by one in a queue, which is called the **run queue**.

There is an inherent trade-off between performance and fairness. A fair scheduler (e.g., RR) evenly divides the CPU among active jobs on a small time scale, at the cost of turnaround time. To be more concise, typically RR has **worse CPU efficiency** than SJF. There are more context switching, and the average wait time and turnaround time are generally longer. However, jobs on a RR scheduler are **more responsive**, so you will not feel that a job freezes because it's on the CPU from time to time.

Most ordinary users run a lot of interactive jobs on modern operating systems. They value **responsiveness** more than CPU efficiency.

Priority scheduling

Each job is assigned a **priority**. The scheduler always chooses the job with the highest priority to run.

Priorities can be static or dynamic.
- **Static priority** means that a job is assigned a fixed priority when it is submitted to the system. Example: a background email process should get a lower priority than a real-time video game process.
- **Dynamic priority** means that a job's priority may be changing throughout its lifetime in the system.

Static priority scheduling

Jobs with the highest priority are scheduled first. They should be **short-lived** to prevent **starvation** of lower-priority jobs. When there is no job with the highest priority left, jobs with the second highest priority will be scheduled, and so on. Note that when a higher-priority job appears, the running job may or may not be preempted.

Limitations:
- High-priority jobs may run for a prolonged period, or even indefinitely.
- Low-priority jobs may starve to death.
- It does not differentiate between CPU-bound and I/O-bound jobs. More specifically, I/O-bound jobs spend most of their time waiting for I/O to complete. When such a job wants the CPU, we had better schedule it immediately to let it start its next I/O request. In that way, I/O requests can proceed **in parallel** with another process actually computing.

Dynamic priority scheduling

There is no standard way to assign priorities dynamically. An example policy can be:
- All jobs start running at Pr. 3 with time slice 30 ms.
- A job is preempted if its time slice is used up or it starts waiting for I/O.
- When a job is preempted, its priority is changed to **the ceiling of its time slice left / 10 ms**.

We are in fact using Round-Robin for each queue. If a process uses up its time slice, then it's priority will change to zero. If a process is blocked to wait for I/O, then its priority will be reassigned based on the time it has already used.

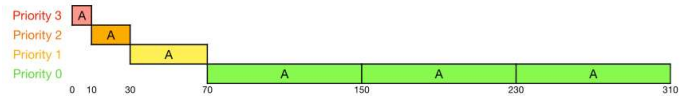
In this way, I/O-bound jobs will have a higher priority to access the CPU.

Multilevel feedback queue (MLFQ)

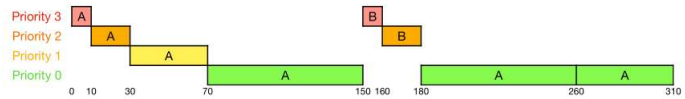
Ideally, we want to:
- Optimize **turnaround** time: run shorter jobs first, and give CPU-bound jobs a larger time slice to reduce context switching.
- Make the system feel **responsive** to interactive users: cannot give all jobs a large time slice, and need to minimize response time.

MLFQ is a kind of dynamic priority scheduling, but **each priority has its own policy**:
- All jobs start running at the highest priority.
- When a job uses up its time slice, its priority is reduced by 1.
- If a jobs gives up the CPU before the time slice is up, it stays at the same priority.

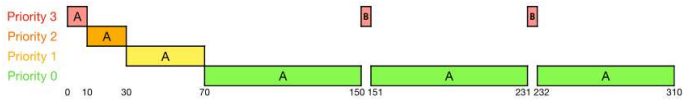
Also, for higher priority, the time slice of round-robin is shorter, and for lower priority, the time slice for round-robin is longer.



Case 1: long-running jobs get a large time slice to reduce context-switching.



Case 2: short jobs run first.



Case 3: I/O-bound jobs have a higher priority to access CPU.

Limitations:
- Long-running jobs may **starve** if there are too many interactive jobs.
- A job may need to run for a long time when it first starts, and becomes interactive after that. Such a job will be punished forever.

Hence, we introduce a new rule: **the priority boost**, which means to move all jobs to the highest priority after some time period.

MLFQ observes how jobs behave over time, and prioritize them accordingly.
- It can deliver excellent overall performance (similar to SJF/PSJF) for short-running interactive jobs.
- It is fair and makes progress for long-running CPU-intensive jobs.

Therefore, many modern operating systems use a form of MLFQ as their base scheduler.

Summary

There is no best or standard algorithm, partly because we cannot predict the CPU requirement of a process, and online scheduling is an NP-hard problem.

Linux employs the **Complete Fair Scheduler (CFS)** since kernel 2.6.23. It is a **dynamic priority scheduling** algorithm based on red-black trees.

Interprocess communication

What is IPC

Processes often need to **communicate** with one another. This is called **interprocess communication (IPC)**. Methods of IPC include but are not limited to: signals; pipelines; file, socket, shared memory; etc.

Why do we need IPC

- To share information.
- To reuse software.
- To speed up computation.
  - Example: MapReduce.
  - You can divide a job into tasks, run them as various processes in parallel, and combine the results.

Case study: piping

Example: *ls | less*
The *pipe()* system call returns two file descriptors:
- *pipefd[0]* refers to the read end of the pipe.
- *pipefd[1]* refer to the write end of the pipe.

来自 <https://man7.org/linux/man-pages/man2/pipe.2.html>
In this case, *ls* produces data into *pipefd[1]* and *less* consumes data from *pipefd[0]*. This is called a **producer-consumer model**.

From the kernel's perspective, if the pipe is full, then *write()* will block, while if the pipe is empty, then *read()* will block. This feature is called **process synchronization**.

Shared memory

Shared memory is a region of memory created by the kernel. It is visible to all processes in the system (by contrast, a pipe is only visible to the two processes at its two ends). Shared memory is also accessible by all processes in the system However, there are syscalls to change the ownership and permissions of the shared memory.

In the case of a pipe, the kernel provides a form of synchronization. However, for shared memory, it is up to the processes to coordinate.

For example, a **race condition** (or, more specifically, a **data race**) may happen, where the results depend on the **timing** of execution, *i.e.*, the particular **order** in which the **shared resource** is accessed. Race conditions are always bad…
- Worse yet, compiler optimizations may generate crazy output if your code has data races. The behaviors are undefined.

Because the computation is **nondeterministic**, debugging is no fun at all. Bugs may disappear or change behavior during debugging.

Mutual exclusion

To avoid race conditions, we need **mutual exclusion**:

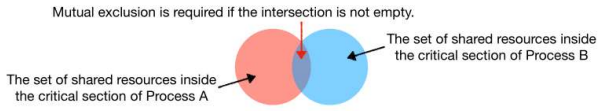
- If one process is accessing a shared resource, the other process must be **excluded** from accessing that same thing.
- Race condition is a problem, and mutual exclusion is a requirement to avoid such a problem. However, mutual exclusion may hinder the performance of parallel computations.

Critical sections

A **critical section** (*a.k.a.* **critical region**) is a piece of code that accesses a shared resource. We declare the entry of the critical region, in which lies the piece of code that accesses the shared resource. After that, we declare the exit of the critical section.

When a process enters its critical section, it **locks** the door. When it exits its critical section, it **unlocks** the door. Different processes may have **different critical sections**.

- A critical section is a piece of code, not a shared resource.
- A critical section should be as tight as possible.
- A critical section may access **multiple shared resources**.

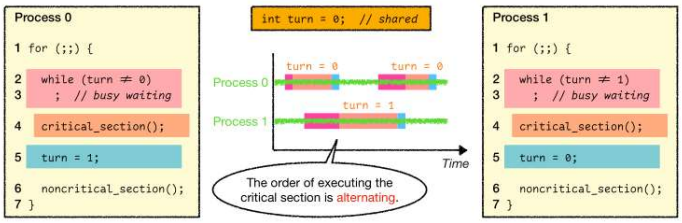


Requirements:
- No two processes may be simultaneously inside their critical sections. This is the mutual exclusion requirement: when one process is inside its critical section, any attempt to go inside the critical sections by other processes are **not allowed**.
- No assumptions may be made about the speeds or the number of CPUs. The solution cannot depend on the time spent inside the critical section or assume the number of CPUs in the system.
- No process running outside its critical section may block other processes. This ensures all processes can make progress. Otherwise, it may end up with a scenario where all processes are blocked but no process is inside its critical section.
- No process should have to wait forever to enter its critical section. This guarantees **bounded waiting**, *i.e.*, no process will starve to death.

Note: It is always the **section entry** that gets blocked, not the critical section itself!

Attempt 1: When one process is in its critical section, it disables interrupts, so that the CPU will not be switched to another process. On single processor systems, this may be correct, but on multiprocessor systems, other CPUs can still access the shared resource, so this is of no use. Also, even on a single processor system, it is a **terrible** idea to allow user processes to enable/disable interrupts at will. However, within the OS kernel itself, it is often convenient to disable interrupts for a few instructions.

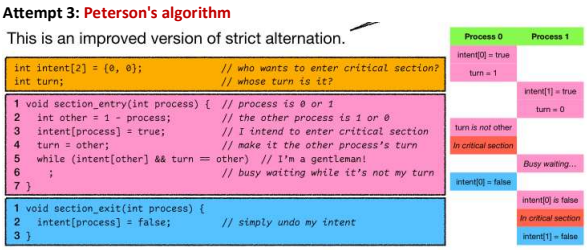
Attempt 2: Strict alternation



This indeed avoids all races. However, what if Process 0 wants to enter its critical section twice in a row? Hence, this is violating the requirement that **no process running outside its critical region may block other processes**.

Attempt 3: Peterson's algorithm

This is an improved version of strict alternation.



Peterson's algorithm is a combination of using **lock variables** and **taking turns**. It is indeed correct on early hardware, where *intent[]* and *turn* propagate immediately and atomically. But it is **not anymore correct** on modern hardware due to relaxed memory consistency models. Also we note that with a little **hardware support**, the solution could be much easier.

Attempt 4: spinlocks

Modern CPUs all have instructions that guarantee **atomic operation**. The simplest one is a **test-and-set** instruction. Conceptually, it behaves as if this code snippet is executed without interruption:

```
int test_and_set(int *ptr, int new) {
    // this code executes atomically
    int old = *ptr;
    *ptr = new;
    return old;
}
```

It returns the old value pointed to by *ptr* and simultaneously updates to *new*.

```
int lock = 0;  // 0: available, 1: held
```

```
void section_entry(int *lock) {

  while (test_and_set(lock, 1) == 1)
    ;  // busy waiting

}
```

```
void section_entry(int *lock) {

  *lock = 0;

}
```

In this code, *test_and_set(lock, 1) == 1* means setting the pointer which is originally 1 to 1 again. If this is true, then another process must be in the critical section. As long as it exits the critical section, setting the value that *lock* points to as 0, immediately some process can set *lock* as 0, escape the while loop for waiting, and enter the critical section.

Spinlocks are built upon CPU instructions that guarantee **atomic operation** (e.g., test-and-set, compare-and-swap, fetch-and-add, load-linked/store-conditional, etc.) It is indeed correct on a **preemptive** scheduler, and beware if the **priority inversion problem**, where a high priority process waits for a low priority process to relinquish resource.

It is indeed a good solution. However, fairness is not guaranteed, since a process may spin forever under contention, leading to starvation. As for the performance, it is bad on single-processor systems, since with *N* processes contending for a lock, *(N-1)/N* of CPU time will be wasted. However, it is effective on multiprocessor systems if the number of processes is approximately the number of CPUs, and the **critical section is short**.

Notice that there may be too much spinning using spinlocks, so we may use *sched_yield()* to give up CPU in the busy waiting while loop. However, while better than spinning, the **run-and-yield** approach is still costly. Before the process holding the lock gets to run again, every other process still needs to run and yield, and note that context switching is expensive.

Attempt 6: semaphores

A **semaphore** is an object with a **non-negative** integer value, which must be initialized before being used. There are two operations: *down()* and *up* (see *man 7 sem_overview* for more information).

```
void down(semaphore *sem) {
  // atomic operation
  if (*sem > 0) {
    *sem = *sem - 1;
  } else {
    block on sem;
  }
}
```

```
void up(semaphore *sem) {
  // atomic operation
  if (some process is blocked on sem) {
    let one such process proceed
  } else {
    *sem = *sem + 1;
  }
}
```

```
// init a binary semaphore
semaphore sem = 1;

down(&sem);

Critical section
  ... ...

up(&sem);
```

In this implementation, a process tries to *down(&sem)*. If *sem* has value 1 then it sets it to 0 and enters the critical section. Otherwise, it is **blocked** on *sem*. Now when a process exits the critical section, it either choose a process blocked on *sem* to proceed into the critical section, or when there is nothing blocked on *sem*, it sets the value to 1.

We just used a **binary semaphore** to implement a **mutex** (mutual exclusion). It is indeed correct, and it is indeed a good solution. Actually, semaphores are more powerful than guaranteeing mutual exclusion. They can be used to realize **process synchronization**, *i.e.*, to coordinate the set of processes so they can produce meaningful output. Note that if the value of a semaphore is greater than 1, we call it a **counting semaphore**.

Classical IPC problems

The producer-consumer problem
- It models access to a bounded buffer.

The dining philosopher problem
- It models processes competing for exclusive access to a limited number of resources (*e.g.*, I/O devices).

The readers and writers problem
- It models access to a database.

The sleeping barber problem
- It models a queueing situation.

The producer-consumer problem

(*a.k.a.* the bounded-buffer problem)

Two processes **share a fixed-size buffer**. The **producer** inserts data to the tail of the buffer. The **consumer** removes data from the head of the buffer. (Yes, it is a pipe.)

Synchronization requirement 1: when the producer wants to insert an item into the buffer, but the buffer is **already full**, what should the producer and the consumer do?
- The producer should **block**.
- The consumer should **wake up** the producer after it has consumed an item.

Synchronization requirement 2: when the consumer wants to remove an item from the buffer, but the buffer is **empty**, what should the producer and the consumer do?
- The consumer should block.
- The producer should **wake up** the consumer after it has produced an item.

Of course, we also need **mutual exclusion**, meaning that no two process can access the shared buffer at the same time.

```
semaphore mutex = 1;            // controls access to critical section
semaphore empty = BUFFER_SIZE;  // counts empty buffer slots
semaphore filled = 0;           // counts filled buffer slots
```

Why do we need three semaphores?

```
1  void producer() {
2    int item;
3
4    for (;;) {
5      item = produce_item();
6      down(&empty);   // decrement empty count
7      down(&mutex);   // enter critical section
8      insert(item);   // put new item in buffer
9      up(&mutex);     // exit critical section
10     up(&filled);    // increment filled count
11   }
12 }
```

```
1  void consumer() {
2    int item;
3
4    for (;;) {
5      down(&filled);    // decrement filled count
6      down(&mutex);     // enter critical section
7      item = remove();  // take item from buffer
8      up(&mutex);       // exit critical section
9      up(&empty);       // increment empty count
10     consume_item(item);
11   }
12 }
```

In the *producer()* function, it first produces an item, and it decrements the number of empty slots if the number of empty slots is not 0 (otherwise it is blocked on *empty*). Next it enters the critical section if there is no process currently in the critical section (otherwise it is blocked on *mutex*). In the critical section, it inserts the item in the shared buffer and then exits the critical section, choosing another process blocked on *mutex* to proceed if exists. Then it increments the number of filled slots if there is no processes blocked on *filled* (otherwise it lets a consumer blocked on *filled* to proceed and do not modify the count. Note that consumers may be blocked on *filled* because the shared memory is empty).

In the *consumer()* function, it decrements the number of filled slots if the number of filled slots is not 0 (otherwise it is blocked on *filled*). Next it enters the critical section if there is no process currently in the critical section (otherwise it is blocked on *mutex*). In the critical section, it removes an item from the shared buffer and then exits the critical section, choosing another process blocked on *mutex* to proceed if exists. Then it increments the number of empty slots if there is no process blocked on *empty* (otherwise it lets a producer blocked on *empty* to proceed and do not modify the count. Note that producers may be blocked on *empty* because the shared memory is full). Finally it consumes the item that it removes from the buffer.

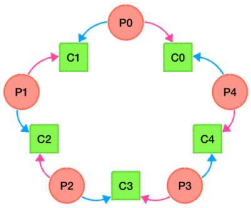
Important remarks:
- *mutex* guarantees **mutual exclusion**.
- *empty* and *filled* are for **synchronization**.
  - *empty* represents the number of **empty slots**. The producer blocks when this value is 0, *i.e.*, there is no empty slot, or in other words, the buffer is already full.
  - *filled* represents the number of **filled slots**. The consumer blocks then this value is 0, *i.e.*, there is no filled slot, or in other words, the buffer is empty.
- **Invariant**: *empty* + *filled* = buffer size.

If we change exchange line 6 and 7 in the *producer()* function, however, it will not work. Say the producer first holds the mutex lock, and then is blocked on empty. Then when the consumer wants to enter the critical section, it will be blocked on mutex since the corresponding lock is held by the producer. Then it will neither be able to increment the empty count, so that the producer will be blocked forever,

and the consumer as well. This scenario is called a **deadlock**. It happens when there is a **circular wait**.

The dining philosopher problem

5 philosophers around a table, 5 instant ramen noodles, and 5 chopsticks in between. A philosopher does only two things in his entire life: **eating** and **thinking**. In order to eat, a philosopher needs exactly two chopsticks.



For each philosopher P[i], his left chopstick is C[i] and his right chopstick is C[(i+1)%N]. His left neighbor is C[(i+N-1)%N] and his right neighbor is C[(i+1)%N]. Each philosopher has three states:

- Thinking
- Hungry (trying to get chopsticks)
- Eating (within critical section)

The random backoff approach

```
void philosopher(int i) {
  for (;;) {
    think();
    take_chopstick(i);
    while (try_to_take_chopstick((i + 1) % N) == FAIL) {
          /* it either succeeds and takes the chopstick,
             or fails and does not take the chopstick. */
      put_chopstick(i);
      sleep(random());    // wait a random time
      take_chopstick(i);
    }
    eat();                // critical section
    put_chopstick((i + 1) % N);
    put_chopstick(i);
  }
}
```

In this approach, we need a semaphore for each chopstick. The functions *take_chopstick(i)* is just to down the corresponding semaphore, and *put_chopstick(i)* is just to up the corresponding semaphore. For each philosopher, after trying to take up one of the chopsticks, it cannot directly try to take up the other one, since if he is not able to take up the other chopstick, he will keep locking the first chopstick he take, causing a **deadlock**. Therefore, he should try to take up the other chopstick, and if he is not able to do so, he puts the first chopstick back, waits for some time, tries to take the first chopstick back, and iterate this process. After eating, the philosopher will just simply release the locks of both chopsticks.

Notice that we need to wait for a **random** amount of time. Otherwise, if there are only two philosophers, and say they will wait for a fixed amount of time 1s if they fail to take the second chopstick. Then they will keep trying and waiting, trying and waiting, in a loop without making any progress. This scenario is called a **livelock**, in which no process is blocked, but they do not make any progress either.

This random backoff approach is good in most applications. In fact, many **network protocols** are designed in this way.

The non-randomized approach

```
int state[N];          // THINKING, HUNGRY, or EATING
semaphore mutex = 1;
semaphore sem[N];      // what are the initial values?
```

```
void take_chopsticks(int i) {
  down(&mutex);
  state[i] = HUNGRY;
  test(i);          // try to take both chopsticks
  up(&mutex);
  down(&sem[i]);  // block if cannot take chopsticks
}
```

```
void put_chopsticks(int i) {
  down(&mutex);
  state[i] = THINKING;
  test(LEFT);   // see if left neighbor can now eat
  test(RIGHT);  // see if right neighbor can now eat
  up(&mutex);
}
```

```
void test(i) {
  if (state[i] == HUNGRY &&
      state[LEFT] != EATING &&
      state[RIGHT] != EATING) {
    state[i] = EATING;
    up(&sem[i]);  // take both chopsticks
  }
}
```

```
void philosopher(int i) {
  for (;;) {
    think();
    take_chopsticks(i);  // section entry
      // block until I take both chopsticks
    eat();               // critical section
    put_chopsticks(i);   // section exit
  }
}
```

In this approach, we assign a semaphore to each philosopher instead of each chopstick. When a philosopher finishes thinking, he will lock the mutex and mark himself as hungry. Then he will try to take both chopsticks. He will succeed if and only if his left and right philosopher are both not eating, and in this case he marks himself as eating and **ups** his semaphore. Otherwise he fails and does nothing. After such trial, he releases the mutex and **downs** his semaphore. Here if he succeeded taking the chopsticks he should have already upped it, and he will just proceed, but if he did not, he will not be able to down the semaphore and will get blocked.

We first see the case when he succeeded taking his chopsticks. Then he just simply eats in the critical section. After that, he will lock the mutex and mark himself back as thinking. Then he will try to test if his neighbors can eat. If they can, he will help them **up** their semaphore, so that they can return from block and eat. After helping his neighbors he will release the mutex, and proceed to the next **thinking-hungry-eating** loop.

In the case that he failed, as is just discussed, he will be blocked on his own semaphore. This will happen if and only if at least one of his neighbors are eating. After his neighbors finishes eating, they will help him return from block, and so that he can proceed to eat.

This non-randomized approach is **deadlock-free**, **starvation-free**, and allows the **maximum parallelism** for any number of processes. Just to summarize, this problem is again about mutual exclusion and synchronization. **Multiple** process compete for **multiple** resources. Each process needs **two distinct** resources to enter the critical section.

The readers and writers problem

Multiple processes are allowed to **read** a database at the same time, but if some process is **writing** to the database, then no other process can have access to it.

Synchronization

- When a reader is reading, other readers are allowed to read the database.
- When a reader is reading, no writers are allowed to write to the database.
- When a writer is writing, no readers or other writers are allowed to access the database.

Concurrency

- Concurrent access from multiple readers should be allowed.

The sleeping barber problem

There is one barber, one barber chair, and *n* chairs for waiting customers. If there are no customers, the barber falls asleep. When a customer arrive, he wakes up the barber. If more customers arrive while the barber is busy, they will wait as long as there are still empty chairs and leave otherwise.

This problem is similar to various **queueing** situations in a **client-server model**. There is one long-running server process (the barber) and many transient client processes (the customers).

```
semaphore barbers = 0;     // # of barbers ready to cut hair (why 0 instead of 1?)
semaphore customers = 0;   // # of customers waiting for service (not being cut)
int waiting = 0;           // same as above (because there's no way to read a semaphore's value)
semaphore mutex = 1;       // controls access to "waiting"
```

```
void barber() {
  for (;;) {
    down(&customers);  // sleep if no customer
    down(&mutex);
    —waiting;
    up(&barbers);  // a barber is ready to serve
    up(&mutex);
    cut_hair();    // outside critical section
  }
}
```

```
void customer() {
  down(&mutex);
  if (waiting < NUM_CHAIRS) {
    ++waiting;
    up(&customers);  // wake up barber if needed
    up(&mutex);
    down(&barbers);  // wait if no barber is free
    get_haircut();   // outside critical section
  } else {
    up(&mutex);      // leave if the shop is full
  }
}
```

It's data-race-free and deadlock-free. Any starvation?

For the barber, if there is no customer he just waits for customers to come, and otherwise he accepts one customer and decrements the waiting count by 1 (this is necessary and should be in the critical section, since the consumers also need to know the waiting count, and there is no way to read the value of a semaphore). Then he **ups** himself, meaning that he is busy, and cuts the hair.

For each customer, he will first access the waiting count (in the critical section). If he finds that there are empty customer chairs, then he increments the customer number (which can wake up the barber from blocked if needed) and also increments the waiting count by 1. Then he exits the critical section and tries to access the barber. If the barber is free the barber will be set busy by **downing**, and otherwise he will be blocked until the barber is free. He gets the haircut then. Otherwise there are no empty customer chairs, then he will just disappointedly exit the critical section, do nothing and leave.

Summary for IPC problems

IPC problems involve:
- One or more **shared** sources;
- **Multiple processes** that must be synchronized.

A good solution need to:
- Guarantee **mutual exclusion**;
- Guarantee proper **synchronization** among processes;
- Be **deadlock-free** and **starvation-free**.

Deadlocks

Conditions for resource deadlocks

- Mutual exclusion: each resource is either available or currently assigned to exactly one process.
- Hold and wait: processes currently holding resources that were granted earlier can request new resources.
- No preemption: resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.
- Circular wait: a **cycle** of 2+ processes, each waiting for a resource held by the next member of the cycle.

If we can attack any one of the above conditions, the deadlocks will be structurally impossible.

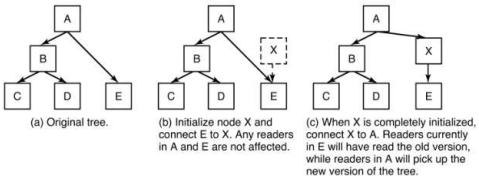
Attack #1: no mutual exclusion

Method 1: make data read only, so processes can use the data concurrently.
Method 2: implement **lock-free** data structures.
- Use powerful hardware instructions to build data structures in a manner that does not require locking.
- We discussed the *test-and-set* instruction when we introduced spinlocks, and another powerful instruction is *compare-and-swap* (CAS):

```
bool compare_and_swap(int *ptr, int expected, int new) {
    // this code executes atomically
    if (*ptr != expected)
        return false;
    *ptr = new;
    return true;
}
```

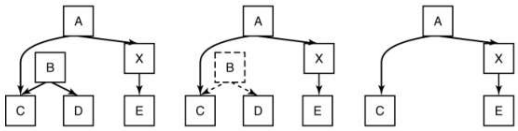
  Using this atomic instruction, we can make sure that it is the current thread/process that modifies the value. Examples include:
  - Atomically increment a value: *do { old = *ptr; } while (!compare_and_swap(ptr, old, old + increment));*
  - Insert a node at the head of a linked list: *do { n -> next = head; } while (!compare_and_swap(&head, n -> next, n));*

Method 3: **read-copy-update (RCU)**.
- We can allow a writer to update the data structure while other processes are still using it.
- A reader would see and traverse **either the old version or the new version**, but not some mash-up.
  - Example: insert a node X in a tree:



(a) Original tree. (b) Initialize node X and connect E to X. Any readers in A and E are not affected. (c) When X is completely initialized, connect X to A. Readers currently in E will have read the old version, while readers in A will pick up the new version of the tree.

  - Example: remove a node B from a tree:



(d) Decouple B from A. Note that there may still be readers in B. All readers in B will see the old version of the tree, while all readers currently in A will see the new version. (e) Wait until we are sure that all readers have left B and D. These nodes cannot be accessed any more. (f) Now we can safely remove B and D

  RCU decouples the **removal** and **reclamation** phases of the update.

Attack #2: no hold-and-wait

Method 1: request all resources at once before starting execution.
- However, a process may **not** know what resources they will need until they have started running.
- Resources are held for longer than needed, thereby **decreasing concurrency**.
- Nevertheless, some mainframe **batch systems** use this approach.

Method 2: release all resources before requesting a new one.

Attack #3: no no-preemption

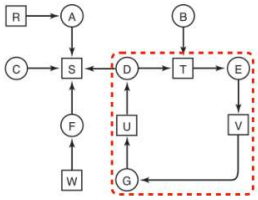
Technically, we cannot forcibly take away a resource that a process already holds. However, in practice, a process can **preempt** their own ownership in a graceful way. We can use **trylock**, as discussed in the dining philosopher problem.

Attack #4: no circular wait

It's probably the most practical and frequently used approach. All the resources are given a **total order** (*i.e.*, global numbering). A process can request only resources **higher** than what it's already holding.

Detecting deadlocks

A deadlock detector runs periodically and build the **resource allocation graph**.



Then DFS is applied to find whether there are cycles in the directed graph.

Threads: lightweight processes

A thread is an **execution entity** within a process. So far, we have only discussed **single-threaded** processes. A **multithreaded process** can have more than one execution in it. For instance, a word processor may have a thread interacting within the user, a thread formatting text in the background, and a thread handling automatic backups.

What is a thread?

- All threads share the same code. A new thread starts a specific **thread function** which can invoke other functions and system calls. However, the thread function does **not** return to its caller.
- All threads **share the same global variables and dynamically allocated memory**. Therefore, a multithreaded process can have **race conditions**.
- Each thread has its own stack of local variables. However, you can still access another thread's stack if you know the memory address.

Why threads?

Threads allow **multitasking** within a process, which can lead to better **performance** and **responsiveness**. Threads are also easier to create an destroy than processes (10--100 times faster). Since threads share the same address space, so sharing data is much easier.

Thread models

- Many-to-one model: implements threads in **user space**.
- One-to-one model: implements threads in the **kernel**.
- Many-to-many model: **hybrid** implementations.

Many-to-one model

The threads library is entirely in user space. The kernel knows that there is a user space, but knows nothing about the threads in there. Such implementation is used in earlier operating systems.

The advantage is that it does not need OS support, and it is fast since it involves no traps, context switches, cache flushing, etc. However, disadvantages of such an implementation include:
- When a **blocking system call** is invoked, all threads will be blocked.
- **Page faults** (discussed later) in a thread will block the entire process.
- **No preemption** of thread due to the absence of clock interrupts.

One-to-one model

The kernel manages all the threads. Each thread is mapped to a **kernel thread**. Such implementation is used in most operating systems.

The advantage is that, when a thread blocks, the kernel can **schedule** another thread (from either the same process or a different process). However, disadvantages of such an implementation include:
- Creating and destroying threads are **more expensive**.
- What if a multithreaded process forks? When a signal comes in, which thread should handle it?

Many-to-many model

The threads library **multiplexes** (多路复用) user-level threads onto kernel threads. It is a hybrid, **two-level** model, and implemented in many language runtimes (Erlang, Go, Haskell, JVM, ...)

This implementation is the best of both worlds (more flexible), and there will be no restrictions on the number of threads. It is just very complex to implement.

POSIX threads *(man 7 pthreads)*

Description	Process	Thread
Process/thread identification.	`pid_t`	`pthread_t`
Get process/thread ID.	`getpid()`	`pthread_self()`
Create a new process/thread.	`fork()`	`pthread_create()`
Terminate the calling process/thread.	`exit()`	`pthread_exit()`
Wait for a specific process/thread to exit.	`waitpid()`	`pthread_join()`
Send a signal to a process/thread.	`kill()`	`pthread_kill()`
Release the CPU to let another process/thread run.	`sched_yield()`	`pthread_yield()`

Note: need to link the *pthread* library, for instance, *gcc -pthread thread.c -o thread*

```
void *greet(void *name) {
  printf("Hello, %s!\n", (char *)name);
  pthread_exit(NULL);
}

int main() {
  pthread_t tid;
  pthread_create(&tid, NULL, greet, "world");
  printf("I am still running...\n");
  pthread_join(tid, NULL);
}
```

thread.c

Mutual exclusion and condition variables in thread programming

A **mutex object** of *type pthread_mutex_t* is similar to a binary semaphore, but it cannot be used as counting semaphores. Locking is equivalent to downing, and unlocking is equivalent to upping.

Condition variables of type *pthread_cond_*t are used instead of counting semaphores for synchronization. They allow a thread to wait for a condition to become true.

- The function *pthread_cond_wait(&cond, &mutex)* **waits on** a condition. The calling thread **must have locked** the mutex. When the thread is about to block, mutex will be unlocked automatically and atomically. When the thread is unblocked, mutex will be blocked again, automatically and atomically.
- The function *pthread_cond_signal(&cond)* **signals** a condition. If some threads are blocked on the condition, at least one thread will then be unblocked. Otherwise the signal will just be lost.

The producer-consumer problem with *pthread* library

```
pthread_mutex_t mutex     = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  not_full  = PTHREAD_COND_INITIALIZER;
pthread_cond_t  not_empty = PTHREAD_COND_INITIALIZER;
```

Bounded buffer

```
void producer() {
  int item;
  for (;;) {
    item = produce_item();
    pthread_mutex_lock(&mutex);
    while (count == BUFFER_SIZE)
      pthread_cond_wait(&not_full, &mutex);
    insert(item);  // put new item in buffer
    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&mutex);
  }
}
```

```
void consumer() {
  int item;
  for (;;) {
    pthread_mutex_lock(&mutex);
    while (count == 0)
      pthread_cond_wait(&not_empty, &mutex);
    item = remove();  // take item from buffer
    pthread_cond_signal(&not_full);
    pthread_mutex_unlock(&mutex);
    consume_item(item);
  }
}
```

Implementing semaphores using mutexes and condition values

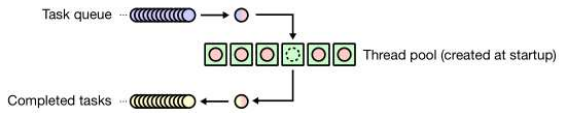
```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cond  = PTHREAD_COND_INITIALIZER;
int value;
```

```
void down() {
  pthread_mutex_lock(&mutex);
  while (value == 0)
    pthread_cond_wait(&cond, &mutex);
  --value;
  pthread_mutex_unlock(&mutex);
}
```

```
void up() {
  pthread_mutex_lock(&mutex);
  ++value;
  pthread_cond_signal(&cond);
  pthread_mutex_unlock(&mutex);
}
```

Thread pool

A **thread pool** is a design pattern for achieving concurrency of execution. It maintains **a pool of worker threads** waiting for tasks to be dispatched. If you have a lot of short-lived tasks, frequent creation and destruction of threads are time-consuming. Using a thread pool increases performance and avoids latency in execution.



Thread safety

Some data structures and functions are designed for single-threaded execution. For example, *strtok()* uses a static buffer while parsing a string, thus **not** thread safe. One way to achieve thread safety is to make the function **reentrant**, so that multiple invocations can safely run concurrently. The function may be interrupted in the middle of its execution and then safely be invoked again before its previous invocations complete. This requires saving state information in variables local to each execution. For example, *strtok_r()* is a reentrant version of *strtok()*, which takes an extra argument that is used internally by *strtok_r()* to maintain context between successive calls that parse the same string.